Amrita Vishwa Vidyapeetham

Real-Time Multi-Core Sorting and Median Filter Service

26ELC331 — Real-Time Operating Systems

Anmol Govindarajapuram Krishnan - CB.EN.U4EEE23103
Mauli Rajguru - CB.EN.U4EEE23120
Sanjay TS - CB.EN.U4EEE23136

October 17, 2026

d.2 Outline

Introduction
System Architecture
Implementation Details
Dataset & File Format
Dedicated Module Testbenches
Two-Fold Verification & Results
Applications & Conclusion

PART 1

Introduction

A client offloads continuous **floating-point time-series** sensor data to a server.

The server must:

- Sort blocks with a **multi-threaded parallel merge sort**,
- Filter sensor outliers with a **rolling median filter**,
- Notify completion asynchronously via a **QNX pulse event**.

The client then collects the calculated median and a memory-offset descriptor.

Real-Time Constraints
High-rate telemetry arrives continuously. Impulsive sensor glitches distort statistical averages and must be removed without stalling collection.

Architectural Rules
QNX synchronous IPC for submission, 2-part IOV transfer ($\geq$ 64 KiB payload), and an asynchronous completion pulse.

d.5 Why a Median Filter Over an Average?

Sensor faults produce high-magnitude impulse spikes. A single faulty sample ruins an arithmetic mean completely:

Window Readings	Mean	Median	Truth
5.0, 5.0, 5.0, 5.0, 5.0	5.0 V	5.0 V	5.0 V
5.0, 5.0, 5.0, 5.0, 50.0	**14.0 V**	**5.0 V**	5.0 V

A 50 V transient corrupts the mean by 9 V, but the median stays at 5 V.

- **Odd kernel width**: Guarantees an unambiguous central sample.
- **Preserving time order**: Median filtering requires chronological order; it must run on a raw copy, not the sorted array.
- **Dual purpose**: Parallel sort gives block percentiles; rolling median cleans live signals.

Rolling median cleanly strips 10–40σ transient spikes without distorting baseline data.

Formal Contract First
A shared header (`protocol.h`) establishes message types, struct layouts, and 64 KiB block boundaries before writing application logic.

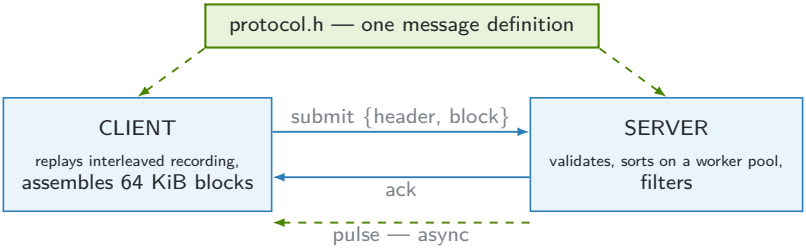
Modular Pipeline
Virtual sensor → block loader → message passing → parallel merge sort. Each module is fully testable on PC and target.

- **Zero padding overhead**: 16 384 float32 samples is exactly 65 536 bytes (64 KiB).
- **Deterministic execution**: Stable merge logic guarantees identical results across 1, 2, 4, or 8 threads.
- **Non-blocking server**: Synchronous ack releases the client immediately; processing continues in worker threads.
- **Empirical rigor**: All performance numbers are measured directly on Raspberry Pi 4 hardware.

PART 2

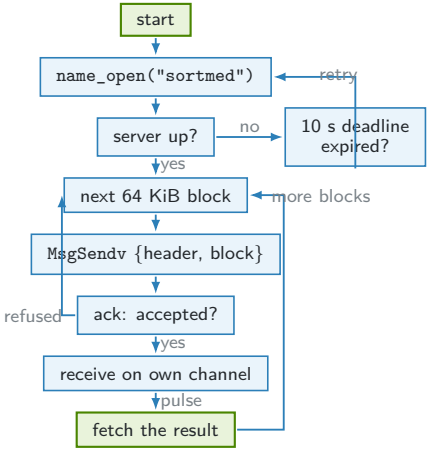
System Architecture

d.8 System overview

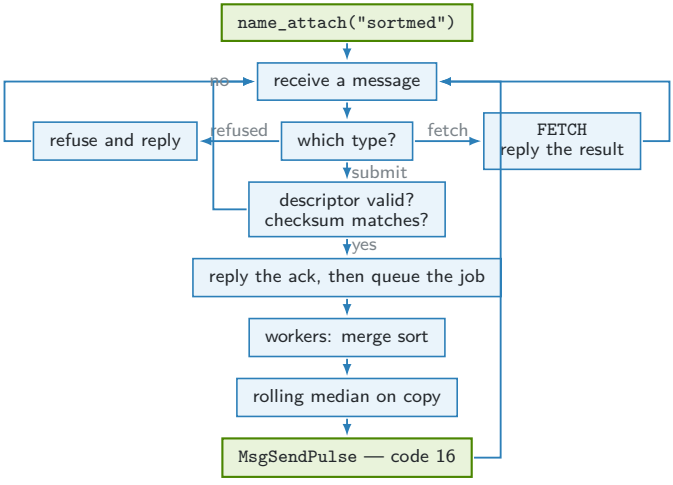


Both programs compile against the same header file. The 64 KiB block crosses as the second part of an IOV transfer: the header is read, validated, and only then is the payload pulled into memory.

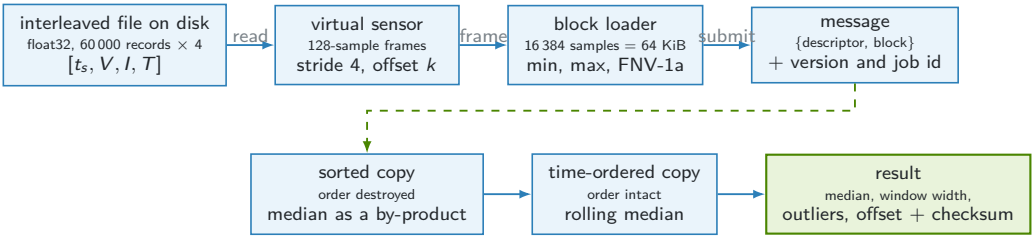
d.9 Client: connect, offload, collect



d.10 Server: accept, validate, answer



d.11 Data flow: what changes at each step



Two buffers, not one. The sort destroys the time order the filter reads, so the arriving block is never sorted in place — its checksum describes it as it arrived.

Every hand-off in this chain is byte-deterministic: fixed-width fields, cleared structs, a zero-padded short block — so any two stages can be compared exactly.

PART 3

Implementation Details

`app/sensor.c` emulates a hardware telemetry device:

- **Production Interleaved Stream**: Reads directly from the interleaved telemetry record (`stride 4, offset k`) with sample formula:

```
idx = (i - win_first) * 4 + offset
```

- **Sliding Window Buffer**: Keeps 4096 records in RAM (`SENSOR_WINDOW_SAMPLES`). Refills on demand via `fread()` as reads advance.
- **Device-Style Frames**: `sensor_read()` delivers at most one frame (default 128 samples).
- **Synthetic Clock**: Exact sample arithmetic: $t = i \times 10^9 / f_{\text{rate}}$ ns.

Strict Input Validation
Validates byte sizes upon opening. Explicitly rejects non-multiple float sizes, zero rates, mismatched record strides, or invalid channel offsets.

Pacing Mechanism
`sensor_read_paced()` compares virtual delivery deadlines against monotonic clock (`clock_gettime` / `nanosleep`) to hold real-world sensor rates.

d.14 Virtual Sensor: Code Walkthrough

```c
/* Bounded window refill & strided extraction */
while (n < cap) {
    if (s->window_count == 0 ||
        s->next < s->window_first ||
        s->next >= s->window_first + s->window_count) {
        if (sensor_fill(s, s->next) != 0)
            return (n > 0) ? (int)n : -1;
    }
    size_t off = (s->next - s->window_first) *
                 s->cfg.stride + s->cfg.offset;
    dst[n++] = s->window[off];
    s->next++;
    s->delivered++;
}
```

- **Sliding Window**: `sensor_fill()` seeks to the required sample and reads a bounded chunk.
- **Zero Memory Bloat**: The full file never resides in memory at once.
- **Exact Stride**: Extracts the chosen channel with fixed stride 4 directly from the interleaved record.

Clean strided extraction without full-file loads

d.15 Block Loader: Aggregation & Serialization

`app/block_loader.c` packages sensor frames into transport parcels:

- **Exact 64 KiB Payload**: Reads frames until `PROTO_BLOCK_SAMPLES` (16 384 floats = 65 536 B) is reached.
- **Interleaved Channel Mapping**: Isolates field ordering in a single function:

  `Voltage` $\rightarrow$ `1`, `Current` $\rightarrow$ `2`, `Temp` $\rightarrow$ `3`
- **Zero-Padded Final Block**: Tail samples of incomplete blocks are explicitly zeroed so uninitialized memory is never transmitted.

Descriptor Metadata
Generates `proto_block_desc_t` for every block:

- Sequence number & first sample index
- Valid sample count
- Min/Max boundaries
- 32-bit FNV-1a checksum

Checksum covers **only declared samples**, not the zero-padding. Guarantees deterministic seals across all runs.

d.16 Block Loader: Code Walkthrough

```c
/* Read full 64 KiB block, pad short tail, seal */
got = sensor_read_block(l->sensor,
                        block->samples,
                        PROTO_BLOCK_SAMPLES);
if (got <= 0) return got;

if ((uint32_t)got < PROTO_BLOCK_SAMPLES) {
    memset(&block->samples[got], 0,
           (PROTO_BLOCK_SAMPLES - got) * sizeof(float));
}
desc->samples  = (uint32_t)got;
desc->checksum = proto_checksum_block(block->samples,
                                      desc->samples);
```

- **Safe Zeroing**: Prevents undefined stack/heap leaks across process boundaries.
- **Reproducibility**: Ensures identical block bytes and checksums on identical data.
- **Server Verification**: The server rejects the submit if the checksum differs by even 1 bit.

Deterministic packing meets the $\geq$ 64 KB rule

d.17 Protocol Definition: Common IPC Header

app/protocol.h defines the shared binary contract:

Message	Size	Description
SUBMIT	48 B	Type, version, job ID, descriptor
BLOCK	64 KiB	16 384 raw float32 samples (IOV)
ACK	8 B	Reply status and assigned job ID
PULSE	1 B code	Code 16, async completion pulse
FETCH	8 B	Client requests result for job ID
RESULT	48 B	Median, offsets, checksums, runtime

Static Assertions (C99)
Checks struct sizes and offsets at compile time: `sizeof(proto_block_t)` is pinned to exactly 65 536 bytes.

Safety and Isolation
Message types start at 0x1000, safely above QNX kernel IO message range (0x100–0x1FF). Pulse code 16 sits in user space (_PULSE_CODE_MINAVAIL).

d.18 Protocol Definition: Shared Results Region

The pulse only says *a job is done*; the arrays are handed over in a **named shared region**, defined in the same header:

Part	Size	Contents
region header	24 B	magic, version, slot count
slot header	16 B	job id, state word
sorted[]	64 KiB	the sorted block
filtered[]	64 KiB	the time-ordered copy

Region `/sortmed-results`: one header plus **4** slots, so four results can be in flight.

A state word, not a lock
The server publishes a slot by writing `WR2D` to its state word. A slot still reading `FREE` has not been written. The client re-reads that one word — no mutex is shared across the programs.

The result points, it does not carry
`proto_result_t` holds a `sorted_offset`, a `filtered_offset`, a `length` and two checksums, all 32-bit. Offset 0 means "not produced".

Three tests pin the region: its name, its layout, and that the result describes it.

`app/mergesort.c` coordinates a POSIX thread pool:

- **Thresholding**: Inputs < 4096 samples run serial merge sort (pool overhead exceeds benefit).
- **Phase 0 (Partitions)**: 64 KiB block splits into k slices ($k = \min(n/1024, \text{threads})$). Workers sort slices concurrently into scratch.
- **Phases** $1 \ldots \log_2(k)$ **(Merges)**: Workers merge adjacent pairs of sorted runs. Buffer pointers swap (ping-pong) between phases.
- **Odd Phase Copy**: If final merge landed in scratch buffer, it is copied back to source in one `memcpy()`.

POSIX Barrier Synchronization
Uses 1 mutex and 2 condition variables:

- `work_ready`: Broadcasts tasks.
- `phase_done`: Signals completion.

No sleeps, no busy-waiting.

Deterministic and Stable
Ties pick from the left run ($a_i \leq b_j$). Equal items preserve order; output is bit-identical across 1, 2, 4, and 8 threads.

d.20 Parallel Merge Sort: Barrier Code Walkthrough

```c
/* Post tasks to workers & wait for phase completion */
static void post_phase(mergesort_pool_t *p, unsigned ntasks) {
    pthread_mutex_lock(&p->lock);
    p->ntasks     = ntasks;
    p->next_task  = 0;
    p->tasks_done = 0;
    pthread_cond_broadcast(&p->work_ready);

    while (p->tasks_done < p->ntasks)
        pthread_cond_wait(&p->phase_done, &p->lock);

    pthread_mutex_unlock(&p->lock);
}
```

- **Manager Thread**: Prepares task list, broadcasts `work_ready`, blocks on `phase_done`.
- **Worker Threads**: Pull next available task index under mutex, execute sort/merge, increment `tasks_done`.
- **Last Finisher**: Signals `phase_done` when `tasks_done == ntasks`.

Strict synchronization without artificial delays

PART 4

Dataset & File Format

d.22 Synthetic Battery Telemetry (PyBaMM)

Realistic multi-channel battery dataset with known ground truth:

- **Simulation**: LG M50 cell model (`Chen2020`) with lumped thermal dynamics, discharged at 0.5C, 1C, 2C.
- **Channels**: Time, Voltage, Current, Temperature (60 000 samples at 100 Hz = 600 s).
- **Noise Injection**: Additive white Gaussian noise (AWGN) + 1.5% impulsive spikes (10–40σ), bursts, and dropouts.
- **Answer Key**: Exact boolean corruption mask preserved offline to score filter accuracy.

Metric	Value
Discharge runs	3 (0.5C, 1C, 2C)
Samples per run	60 000
Channels per sample	4
Total tensor values	720 000
Raw payload size	6.2 MiB (44 blocks)
Corrupted cells	8 040 (1.489%)

Predictable, reproducible testbench for sorting and outlier rejection.

d.23 Format Clarification: Planar vs Interleaved

Planar Format (Single Channel File)
One flat file per signal (e.g. `r1_voltage.bin`). 60 000 contiguous floats (240 000 B).
$\text{sample}_i = \text{file}[i]$ (`stride 1, offset 0`).

Interleaved Format (Production Payload)
One unified record file (`Chen2020_c1...bin`). 60 000 records of 4 floats (960 000 B): $[t_s, V, I, T]$.
$\text{sample}_i = \text{file}[4i+k]$ (`stride 4, offset k`).

Why the Distinction Matters in Code
Sorting planar data directly is meaningful. On an interleaved file, sorting without selecting channel offset k sorts seconds against volts against amperes!

- `block_loader_channel_offset()` isolates this knowledge (Voltage is offset 1, not 2).
- Planar was used only to cross-check offsets during initial testing.
- **Interleaved is our sole production format.**

File: `Chen2020_c1_T25_discharge.bin` (960 000 B)

Each time instant forms a 16-byte record of 4 float32 values:

Offset k	Bytes	Channel	Unit
0	0–3	Time	Seconds (s)
1	4–7	**Voltage**	Volts (V)
2	8–11	Current	Amperes (A)
3	12–15	Temperature	Kelvin (K)

60 000 samples → **4 transport blocks**: 3 full 64 KiB blocks + 1 short block (10 848 samples).

Direct Struct Mapping
Each record matches: `struct { float t, v, i, temp; }` Seek straight to frame n at byte $16n$.

Deterministic Checksums
Extracting the voltage channel ($k = 1$) yields reproducible block checksums:

- Block 0: `0xe90d24d2`
- Block 1: `0xc98cd3b2`
- Block 2: `0x14c728bb`
- Block 3: `0x362ab592`

PART 5

Dedicated Module Testbenches

d.26 Testbench: Virtual Sensor (test_sensor)

1. What it does
Validates telemetry replay fidelity: bounded memory windows, strided extraction, frame boundaries, clock calculation, and error rejection.

2. How it's done

- Writes synthetic ramps, partial floats, and malformed files to test edge rejection.
- Tests bit-exact round trips (± 0.0, denormals, float extremes).
- Verifies strided multi-channel reads against single-sample reads.

3. Verifiable Output

- Rejection diagnostics emitted for invalid inputs (e.g. 3-byte partial floats).
- Synthetic ramp verification: 100% bit-exact match across window refills.
- **15 tests, 642 checks passed (0 failed, 0 skipped).**

`Guarantees sensor fidelity before loading data`

d.27 Testbench: Protocol Definition (test_protocol)

1. What it does

Enforces binary contract invariants: struct memory layouts, 64 KiB block boundaries, message type ranges, and checksum integrity.

2. How it's done

- Asserts `sizeof(proto_block_t) == 65536`.
- Asserts all message IDs reside above QNX system types ($> 0x1FF$).
- Mutates single bits in blocks to verify FNV-1a checksum sensitivity.

3. Verifiable Output

- Zero padding byte determinism confirmed.
- Single-sample corruption immediately flagged as `PROTO_ERR_CHECKSUM`.
- Short final blocks validated on declared sample counts.
- **12 tests, 137 checks passed (0 failed, 0 skipped).**

`Guarantees ABI compatibility between client and server`

d.28 Testbench: Block Loader (test_block_loader)

1. What it does
Validates translation of raw sensor streams into calibrated 64 KiB transport blocks and verified descriptors.

2. How it's done

- Feeds synthetic 20 000-sample stream to verify exact splitting: 16 384 + 3616.
- Verifies zero-padding of the short tail in RAM.
- Validates channel offset mapping on multi-channel records.

3. Verifiable Output

- Reassembly matches input stream bit-for-bit.
- Checksum validates correctly via `proto_submit_verify()`.
- Real 60 000-sample dataset splits into 3 full blocks + 1 short block (10 848).
- **8 tests, 116 checks passed (0 failed, 0 skipped).**

`Guarantees reliable 64 KiB payload packaging`

d.29 Testbench: Parallel Merge Sort (test_mergesort)

1. What it does
Validates multi-threaded sorting correctness, POSIX barrier synchronization, thread invariance, and stability.

2. How it's done

- Compares multi-threaded output against standard libc `qsort`.
- Tests edge boundaries (0 to 64 samples, serial threshold 4096, partition transitions).
- Tests pathological patterns: all-equal, presorted, reverse-sorted, denormals, ± 0.0.

3. Verifiable Output

- Bit-identical results across 1, 2, 4, and 8 worker threads.
- Stable sorting preserves arrival order of duplicate keys.
- Correct dynamic phase task plan reported: 4 partitions, 2 merge phases, 7 tasks.
- **13 tests, 578 checks passed (0 failed, 0 skipped)**.

`Guarantees multi-core scaling without race conditions`

PART 6

Two-Fold Verification & Results

Testing is executed in a rigorous **two-fold manner**:

Fold 1: Host-Side Unit Testing

- Standard GCC/POSIX testbench off the board.
- Tests edge boundaries (0 to 64 samples, ± 0.0).
- Catches 1-bit checksum tampering.
- Asserts strided record extraction.
- Verified against standard `qsort`.

1473 checks, 0 failed, 0 skipped.

Fold 2: On-Target QNX Execution

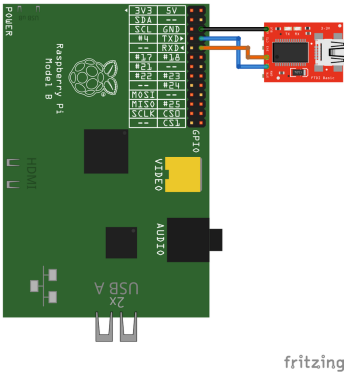
- Cross-compiled with `qcc -Vgcc_ntoaarch64le`.
- Embedded inside boot IFS image (`ifs-rpi4.bin`).
- Bare-metal Raspberry Pi 4 (Cortex-A72).
- Real-time multi-core scaling (1, 2, 4 cores).
- Direct IFS payload file processing.

100% verified pass rate on physical hardware.

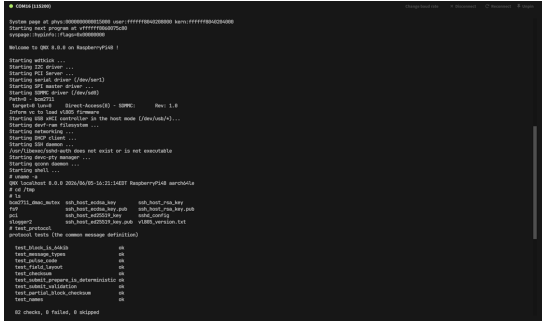
Target Testbench Setup

Hardware Target
Raspberry Pi 4 Model B with 4 Cortex-A72 cores running QNX 8.0.0. Booted directly from SD card into RAM IFS image.

Zero Network Dependency
Controlled strictly via FTDI USB-to-serial adapter on GPIO 14/15 (`/dev/ser1`) at 115200 8N1. Self-contained and noise-free.

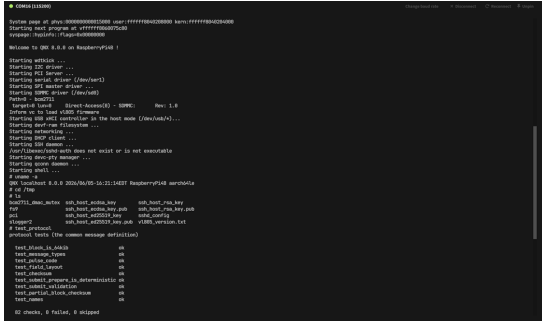


d.33 Console: Board Boot and Initialization



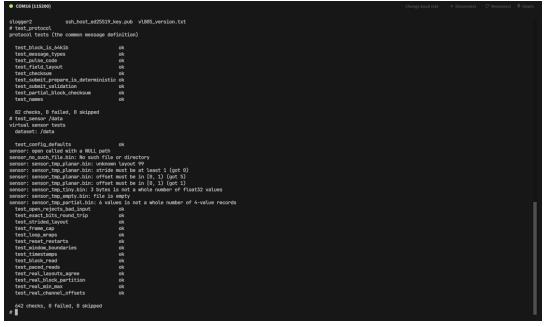
Raspberry Pi 4 boot sequence under QNX 8.0.0. All four Cortex-A72 cores are brought online by `procnto-smp-instr`. System shell initializes on serial console.

d.34 Console: Protocol Layer Verification



`test_protocol` executing on target. Pins the exact 64 KiB block size, application message ID ranges, pulse codes, FNV-1a checksum, and the shared results region.

d.35 Console: Virtual Sensor Verification



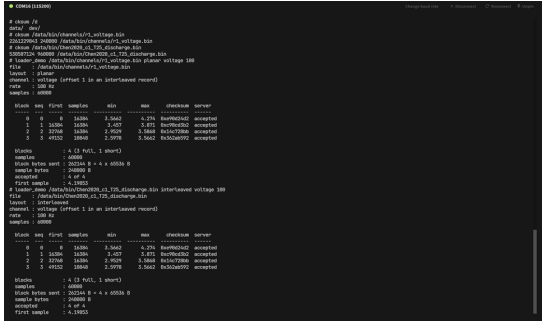
`test_sensor /data`: 642 checks passed. Console verifies robust error trapping: explicitly naming and rejecting invalid strides, bad offsets, and unaligned files.

d.36 Console: Block Loader Verification



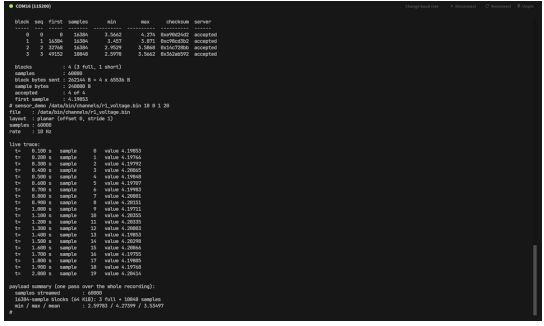
`test_block_loader /data`: 116 checks passed. Confirms strided record extraction produces deterministic 64 KiB blocks and detects bit corruption.

d.37 Console: On-Board Payload Verification



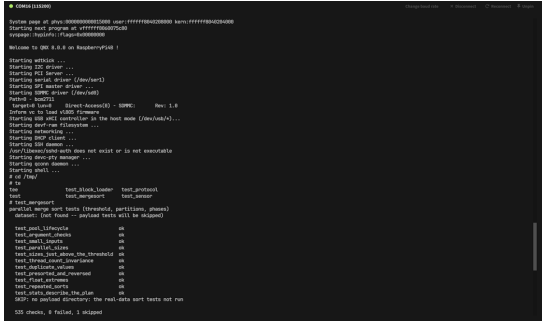
Direct `cksum` comparison of embedded IFS files, followed by `loader_demo` reading the interleaved file. Checksums match host dataset; blocks 0–3 accepted.

d.38 Console: Live Sensor Streaming Demo



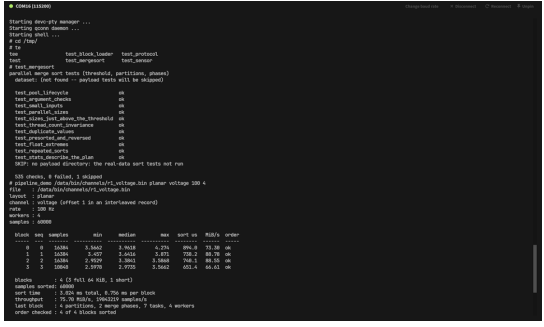
sensor_demo streaming at 10 Hz with hardware monotonic pacing. Reads sample-by-sample, aggregating 60 000 readings into 4 structured blocks.

d.39 Console: Parallel Sorting Verification



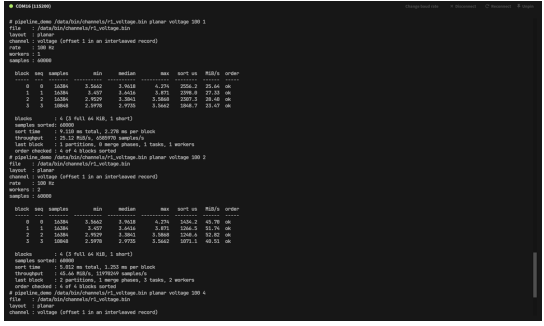
`test_mergesort`: 578 checks passed. Tests edge boundaries (0 to 64 samples), duplicate values, reverse sequences, and thread invariance.

d.40 Console: Real-Time Multi-Core Execution



`pipeline_demo` on real voltage telemetry with 4 workers. Phase plan dynamically reflects load: **4 partitions, 2 merge phases, 7 tasks**.

d.41 Console: Core Scaling Sweep



Worker sweep (1, 2, 4 threads) sorting 64 KiB blocks. Execution time drops from 2.278 ms $\rightarrow$ 1.253 ms $\rightarrow$ 0.757 ms. Medians and ranges remain perfectly identical.

d.42 Benchmark Analysis and Throughput

Workers	Time/Block	Throughput	Speed-up
1	2.278 ms	25.12 MiB/s	1.00×
2	1.253 ms	45.66 MiB/s	1.82×
4	**0.757 ms**	**75.55 MiB/s**	**3.01×**

Measured on physical Raspberry Pi 4 running QNX 8.0.0 SMP.

Massive Real-Time Headroom
One 64 KiB block represents **163.84 seconds** of 100 Hz sensor telemetry. Sorting completes in **0.757 ms** — over **216 000×** real-time speed.

- Excellent parallel efficiency (3.01× scaling on 4 physical cores).
- Negligible thread barrier overhead.

PART 7

Applications & Conclusion

- **Battery Management Systems (BMS)**: High-voltage battery packs generate noisy current/voltage transients during switching. Median filtering suppresses noise while parallel sorting evaluates cell distributions.
- **Industrial Edge Gateways**: Embedded telemetry hubs aggregate thousands of sensor channels; offloading 64 KiB blocks to worker threads keeps acquisition tasks responsive.
- **Automotive Telemetry (ADAS)**: Mission-critical sensor feeds (radar, wheel speed) require deterministic, reproducible sorting across bus channels.
- **Flight Data Recorders**: Summarizing real-time telemetry into verifiable, checksummed statistical descriptors.

> Directly applicable wherever high-speed sensor streams require outlier rejection and rapid percentile extraction.

- **Milestone 1 (Complete)**: 64 KiB block architecture, production interleaved dataset, protocol contract, and virtual sensor.
- **Milestone 2 (Complete)**: Multi-threaded parallel merge sort with POSIX mutex and condition variable phase barriers; verified on target.
- **Two-Fold Verification**: 1473 host checks + bare-metal target suite with 100% pass rate.
- **Performance**: 0.757 ms sort time per 64 KiB block (3.01× speedup on 4 cores).
- **Upcoming Milestones**:
  - M3: QNX client–server IPC messaging and rolling median filter implementation.
  - M4: Asynchronous pulse event delivery and end-to-end timing validation.

Questions and Discussion

THANK YOU

Real-Time Multi-Core Sorting and Median Filter Service

Anmol Govindarajapuram Krishnan - CB.EN.U4EEE23103
Mauli Rajguru - CB.EN.U4EEE23120
Sanjay TS - CB.EN.U4EEE23136

26ELC331 - Real-Time Operating Systems